51. Agents and Tools: Function-Calling Loops

51. Agents and Tools: Function-Calling Loops

Phase: Phase 7 — AI engineering in Go
Estimated time: 2 days
Slug: `51-agents-tools`

Learning objectives

- Implement tool schemas
- Run a bounded agent loop
- Sandbox dangerous tools
- Enforce max steps and budgets

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Agent loop

```
for step := 0; step < maxSteps; step++ {
  decision := llm.Next(history)
  if decision.Type == "final" { return decision.Answer }
  result := tools.Run(decision.Tool, decision.Args)
  history = append(history, result)
}
```

Safety

- Allowlist tools
- Validate args against JSON schema
- Timeouts per tool
- Human approval for side-effecting ops
- Hard caps on steps/tokens/cost

Go's type system and explicit errors help keep tool runners boring and safe.

Common mistakes (especially from Python instincts)

- Unbounded loops
- Executing shell from model output unchecked
- Trusting model JSON without validation

Exercises

Work in `exercises/51-agents-tools/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/51-agents-tools/` until you have an honest failing attempt.

Self-check

1. Why maxSteps?
2. What to validate before tools.Run?
3. Allowlist purpose?

Further reading

- https://go.dev/blog/error-handling-and-go